



TDC
FLORIPA

Ambiente Livre
Big Data & Data Science



Arquiteturas de Streaming com Apache Flink: quando reduzir ou eliminar o uso de Apache Kafka.

Marcio Junior Vieira
CEO & Data Scientist, Ambiente Livre

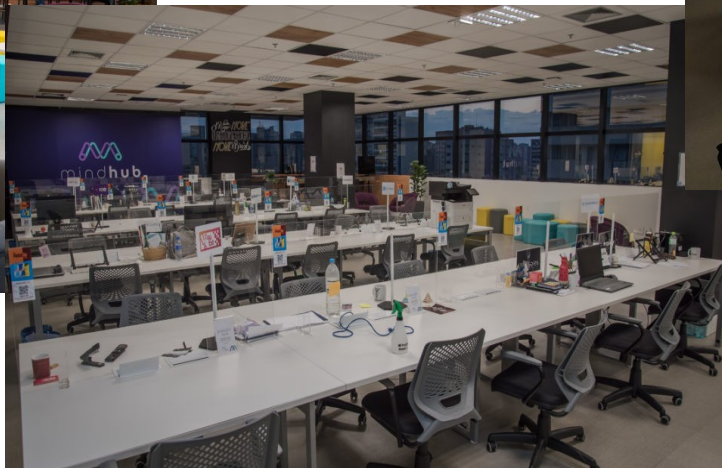
Marcio Junior Vieira

- 26 anos de atuação na área.
 - **CEO** da Ambiente Livre atuando como **Cientista de Dados, Engenheiro de Dados** e Arquiteto de Software.
 - Como **Professor** atuou nos MBAs em Big Data & Data Science, Inteligência Artificial e Business Intelligence da Universidade Positivo e no MBA Artificial Intelligence e Machine Learning da FIAP.
 - Desenvolveu trabalhos como **pesquisador** pela UNB no Laboratório de Tecnologias da Tomada de Decisão - UNB/LATITUDE e pela UFG no CIAP - Centro de Colaboração Interinstitucional de Inteligência Artificial Aplicada às Políticas Públicas.
-
- **10 anos de TDC**



Open Software for Business

- Consultoria com Software Livre e Open Source
- Experts em 34 soluções para geração de negócios com Software Livre/Código Aberto.
- Atualmente estamos sediados no Hub de Inovação Mindhub em Curitiba.



Nosso Ecossistema de Serviços

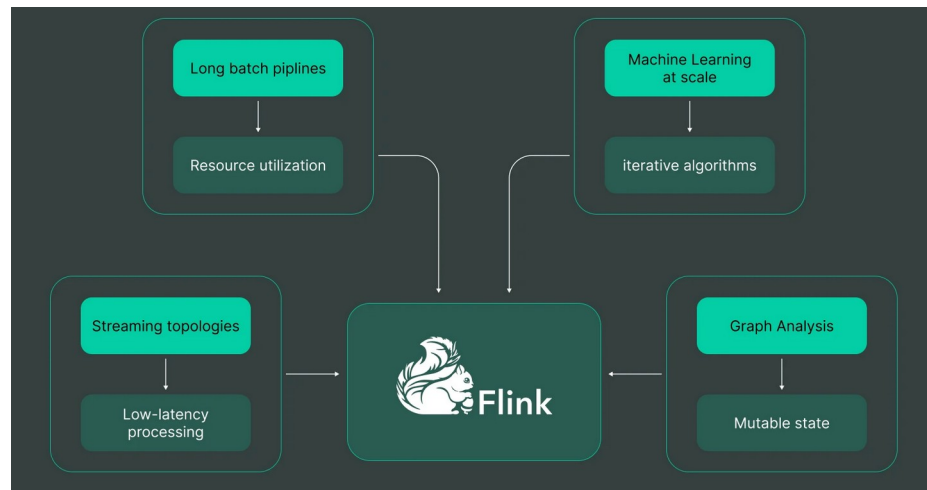
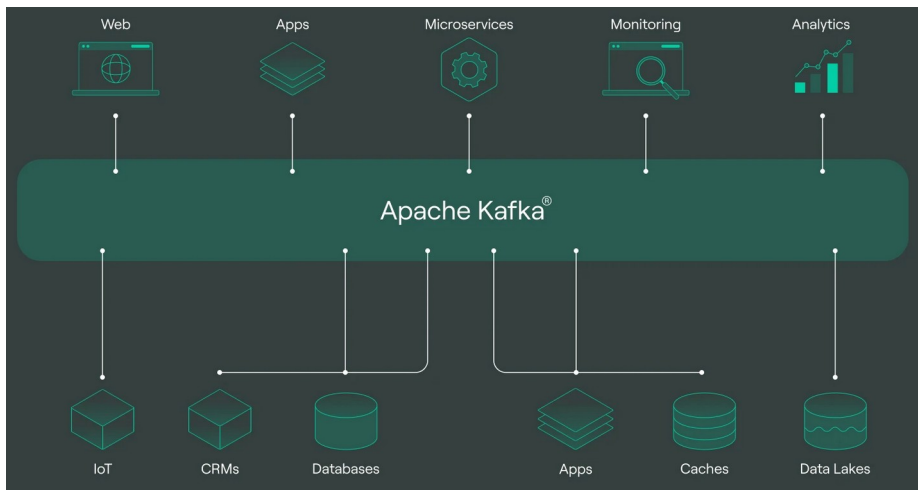
Data Driven	CRM, CMS e ITSM	ECM e BPM	Infra - IAC - DevOps
Painéis de Indicadores Cubos e Relatórios Análise Preditiva Processamento Distribuído Banco de Dados Colunares Dashboards e OLAP Data Integration e Data Mining Big Data & Data Lake Machine Learning Business Intelligence & Analytics Consultoria Treinamento Projeto	Help Desk e Inventory Pesquisas Online Marketing e Vendas SAC e Pós-vendas Portais de Conteúdo IT Service Management Customer Relationship Management Content Management System Content Management Framework EAD e LMS Consultoria Treinamento Projeto	Gestão de Documentos Gerenciamento de Mídias Processo de Negócio BPMN e BPMS Microserviços Enterprise Content Management Records Management Business Process Management Microservices Orchestration Consultoria Treinamento Projeto	DepOps DevSecOps MLOps e DataOps Native Cloud Distributed Systems Web Server Kubernetes-as-a-Service Object Storage Containers Building Blocks Consultoria Treinamento Projeto



Kafka vs Flink: O Encanamento e a Fábrica

Conceito Base

- Log Distribuído/Armazenamento (Kafka) e Engine de Computação Stateful (Flink).



Kafka com Flink



TOPIC: logs

{ "timestamp": "2025-06-07T10:15:00Z",
"level": "INFO", "msg": "User login" }

{ "timestamp": "2025-06-07T10:15:01Z",
"level": "WARN", "msg": "Disk usage high" }

{ "timestamp": "2025-06-07T10:15:02Z",
"level": "ERROR", "msg": "Failed to connect" }

...

Partition 0

Partition 1

Partition N



Stream
of Events



Apache
Flink®

Stream Processing Engine



Ingest & Parse

Consume logs from Kafka



Process

Filter, Enrich, Aggregate



Compute

Real-time Analytics



Stateful Processing

Maintain State



Output

Send results to sinks



Processed
Results

SINKS / OUTPUTS



Data Lake

(S3 / MinIO)



Data Warehouse

(ClickHouse / Druid)



Elasticsearch

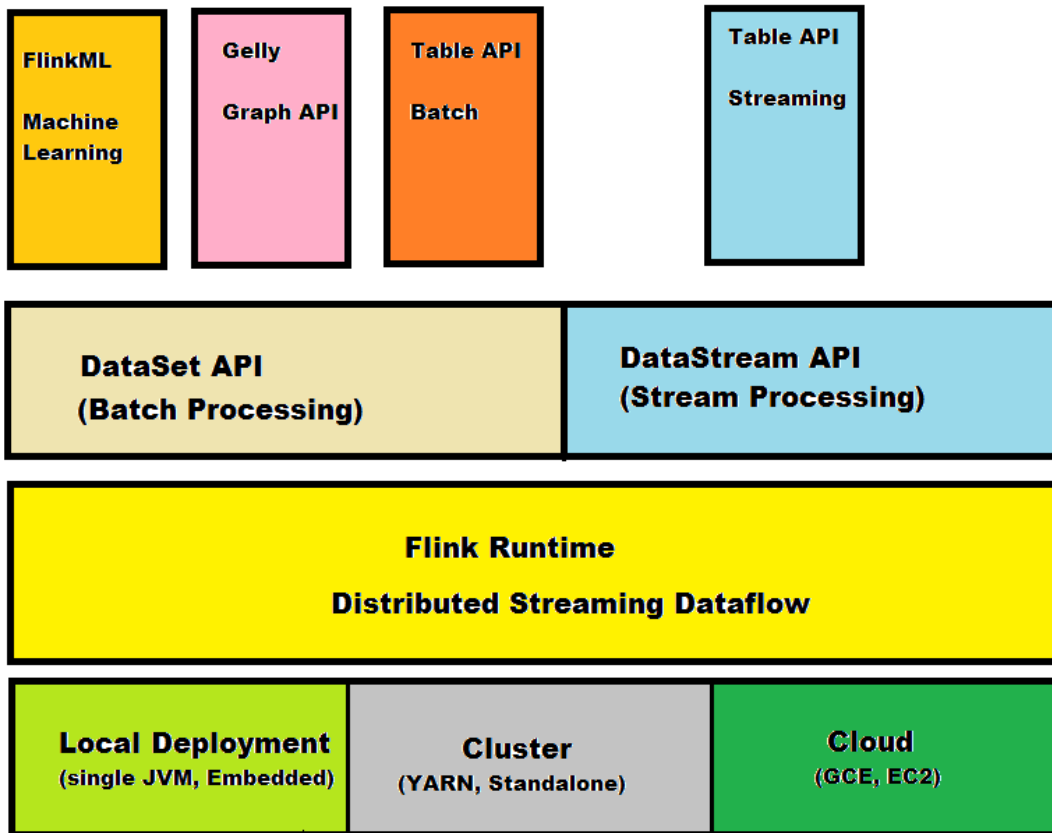


Prometheus

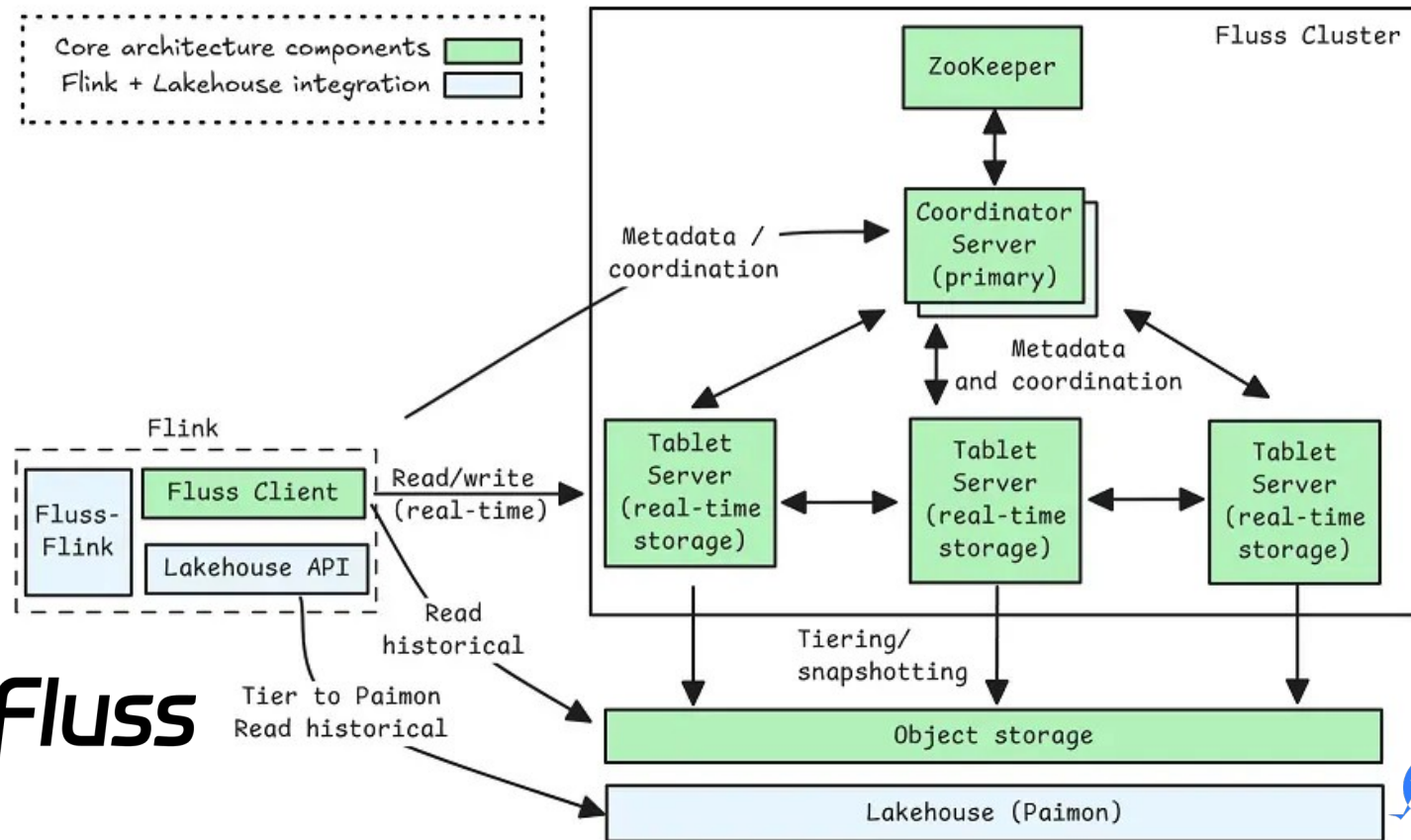
...

More Sinks...

Arquitetura



Flink + Fluss + Paimon

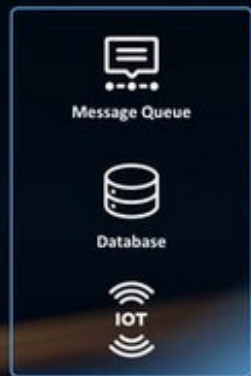


Fluss

Tier to Paimon
Read historical

Real Time LakeHouse / Streaming Lakehouse

External Data Sources



Streaming Lakehouse



Flink

- **Detecção de fraude:** Analisa dados de streaming de transações com cartão de crédito para identificar e detectar atividades fraudulentas, fornecendo insights em tempo real.
- **Detecção de anomalias:** processa dados de streaming de sensores, permitindo identificar anomalias no comportamento do equipamento e tomar medidas proativas.
- **Mecanismos de recomendação:** Analisa dados de streaming de interações do usuário, permitindo que você crie sistemas de recomendação personalizados para produtos ou serviços.





Kafka

- **Sistema de mensagens:** o Kafka facilita a troca de mensagens em tempo real entre aplicativos, permitindo que você envie notificações instantâneas aos usuários assim que elas ocorrerem.
- **Streaming de eventos:** Esta plataforma permite o streaming contínuo de eventos de um sistema para outro, tornando-o ideal para cenários como streaming de dados de sensores de uma fábrica para uma plataforma de análise baseada em nuvem.
- **Agregação de log:** pode agregar logs de várias fontes, como servidores web, tornando-o valioso para solução de problemas e monitoramento de problemas de desempenho



Onde as Arquiteturas Tradicionais Falham?

- Proliferação de tópicos intermediários, latência acumulada por hops de I/O e sobrecarga operacional do cluster Kafka.



State Management

- Como o estado em memória (RocksDB/Heap) local permite agregações e joins complexos sem depender do Kafka como buffer intermediário.



Baixa Latência

Acesso local ao estado em memória para processamento em tempo real.



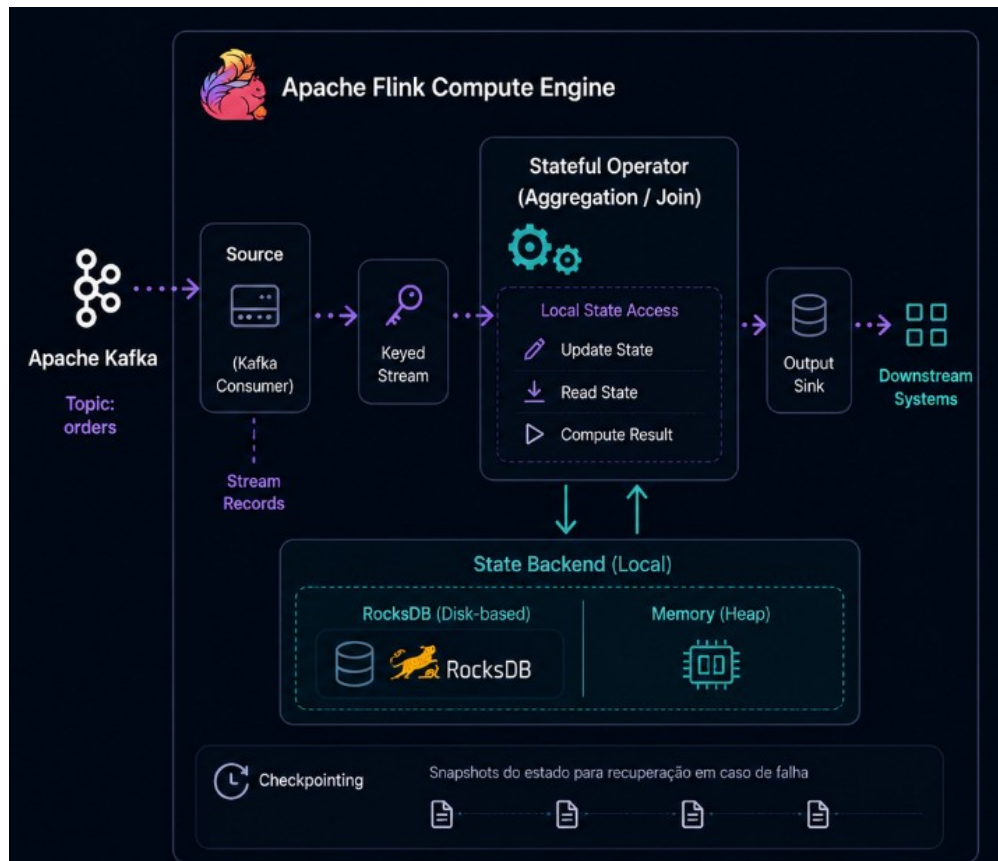
Estado Gerenciado

Flink gerencia o ciclo de vida, checkpoint e recuperação do estado automaticamente.



Alta Confiabilidade

State Backend com RocksDB garante durabilidade e tolerância a falhas.



Escala Extrema no Mundo Real

- Volumes na escala de Terabytes de estado em memória com throughput contínuo sem gargalos de rede.
- **Abordagem tradicional:** Se chegam 100.000 eventos por segundo, você faz 100.000 requisições de rede por segundo. O gargalo migra para a placa de rede e a latência de I/O.
- **Flink (Local State Management):** O Flink armazena o estado do processamento na própria máquina onde o cálculo acontece (usando Heap ou RocksDB embarcado no SSD/NVMe do TaskManager).
- **Vantagem:** A consulta e a gravação do estado acontecem em milissegundos localmente. A rede não é para consultar o estado durante o fluxo do evento.

Latência Fim-a-Fim por Arquitetura

- Comparação do tempo de resposta entre pipelines:
 - ✓ Multi-Hop no Kafka (~120ms),
 - ✓ Híbridos (~35ms)
 - ✓ Ingestão Direta via Flink CDC (~8ms).

Desempenho

- **Flink:** Mais performático do que o Kafka para aplicativos de análise de streaming.
- **Kafka:** Mais performático do que Flink para aplicações de agregação de log. tem uma taxa de transferência maior do que o Flink, o que significa que pode lidar com mais dados por segundo.

Processing Time (Tempo de Processamento):

- É o relógio da máquina onde o Flink está rodando.
- **O problema:** Se a rede do usuário cair e ele enviar uma transação Pix feita às 14:00 somente às 14:15, o sistema que usa Processing Time vai achar que a transação aconteceu às 14:15, corrompendo a análise de fraude.

Event Time (Tempo do Evento):

- É o carimbo de data/hora (timestamp) de quando o fato realmente aconteceu na origem (ex: o momento em que o usuário clicou no app ou o sensor do carro registrou a temperatura).

Watermarks (O Relógio Inteligente do Flink):

- É o mecanismo que avisa ao Flink até que ponto o tempo avançou.
- Diz ao motor: "Nenhum evento com timestamp anterior a X deve chegar mais. Podemos fechar esta janela de tempo de 5 minutos e emitir o resultado."

A Diferença Pragmática das Semânticas:

- **At-Most-Once:** Se a máquina cair, dados são perdidos.
- **At-Least-Once:** Se a máquina cair, dados são reprocessados (pode haver duplicatas, perigoso para sistemas financeiros).
- **Exactly-Once:** O estado final da aplicação é exatamente idêntico ao que seria se nenhum erro tivesse ocorrido na infraestrutura.

O Segredo Tecnológico: Checkpointing Assíncrono

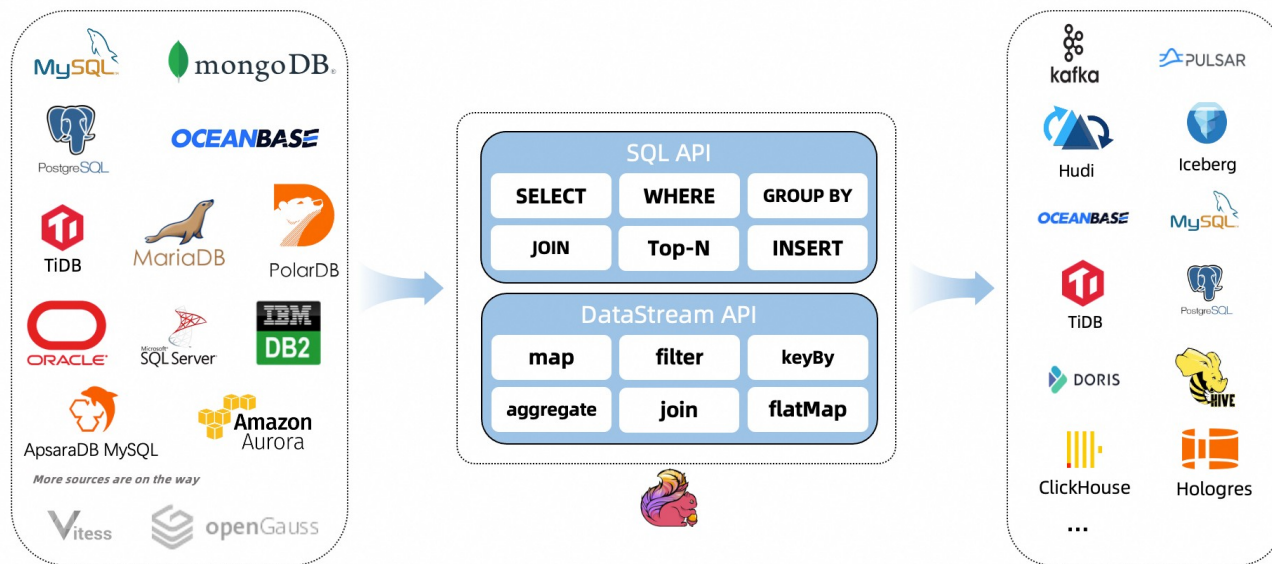
- Em vez de parar o streaming para tirar um "backup", o Flink injeta barreiras de Checkpoint (Checkpoint Barriers) diretamente no fluxo de dados.
- As barreiras fluem junto com as mensagens normais. Quando uma barreira passa por um operador, ele tira uma "foto" (snapshot) do seu estado local de forma assíncrona e não-bloqueante.

End-to-End Exactly-Once (Garantia Ponta a Ponta):

- Não basta o Flink ter o estado interno correto; o dado gravado no destino (S3, ClickHouse, PostgreSQL) também não pode duplicar.
- **O Flink resolve isso combinando:** Source Re-spooling (Kafka/CDC voltando offset) + Internal State Snapshots + Two-Phase Commit (2PC) Sinks.
- **Consistência Estrita de Estado:** Garantia de que cada evento afetará o estado interno da aplicação exatamente uma vez, mesmo diante de falhas de hardware.
- **Snapshots Leves e Assíncronos:** Uso do algoritmo de Chandy-Lamport com barreiras no fluxo de eventos — zero pausa no processamento.
- **Garantia Ponta a Ponta (End-to-End):** Integração transacional em duas fases (2PC) entre a Origem (Source), o Processamento (State) e o Destino (Sink).

Conectores Diretos do Flink

- Integradores nativos como Flink CDC, Cloud Storage (S3/Delta Lake) e Sinks Analíticos (ClickHouse, Pinot, Elasticsearch).



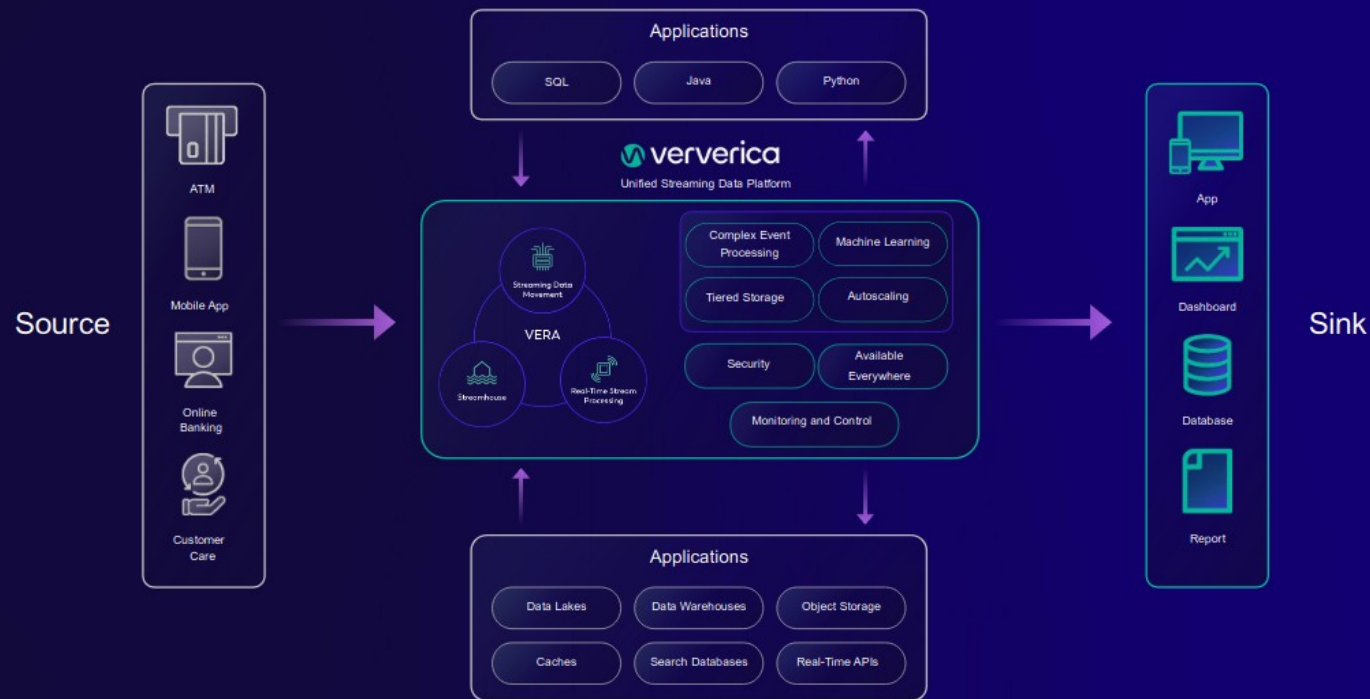


Microsoft



Plataforma Enterprise (Ververica)

Ververica Platform



Contribuições - Open Source - Ververica



Apache Fluss

Next-Gen Streaming
Storage



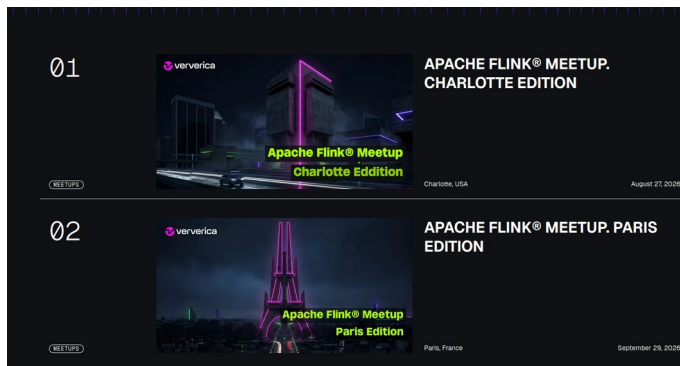
Apache Flink

Stream Processing
Engine



Apache Paimon

Streaming
Lakehouse



Consolide o Flink

- **Etapas Intermediárias de Transformação:** Limpeza, filtragem, parsing de JSON, enriquecimento e junção de dados (Joins). NUNCA crie um tópico Kafka intermediário apenas para passar o dado de um job para outro.
- **Manutenção de Estado Complexo:** Agregações por usuário, janelas temporais (Tumbling/Sliding), deduplicação e controle de sessão em tempo real.
- **Cálculo de Regras e Modelos (ML):** Motor de pontuação antifraude, cálculo de ETA ou sistemas de recomendação operando em milissegundos.

Reduzir

- **O Problema:** Um job Flink lê do Tópico A, faz um filtro, grava no Tópico B. Outro job Flink lê do Tópico B, faz uma junção, e grava no Tópico C.
- **A Solução:** Mantenha o Kafka na Entrada (Event Bus de ingestão) e na Saída (Barramento de notificação/saída). Elimine todos os tópicos intermediários (B, C, D). Faça a composição, joins e agregações na topologia Stateful do Flink.
- **Resultado:** Redução massiva de latência (de ~120ms para ~35ms), economia brutal de custo de rede e disco no cluster Kafka.

Eliminar

- **CDC Direto (Flink CDC):** Quando o objetivo é apenas replicar alterações de um banco relacional (PostgreSQL, MySQL, etc) para um Data Lake ou banco analítico em tempo real. O Flink CDC lê diretamente os logs da base (WAL/Binlog) e escreve no destino, sem necessidade de implantar um cluster Kafka Connect + Kafka Brokers.
- **Fontes Leves / Sem Necessidade de Replay Longo:** Quando a fonte de dados é uma fila de curta retenção (RabbitMQ, SQS) ou WebSockets/HTTP endpoints onde o desacoplamento de múltiplos consumidores e a retenção de dias/semanas não são requisitos.

“Não elimine o Kafka por dogmatismo. Elimine o Kafka quando perceber que o Flink pode fazer o trabalho com metade da infraestrutura e uma fração da latência.”

Marcio Junior Vieira

Sites

- Apache Kafka vs. Flink - **Choosing the right streaming data platform**
<https://double.cloud/blog/posts/2023/06/kafka-vs-flink.html>
- **Apache Flink Documentation** <https://flink.apache.org/documentation/>
- **Apache Kafka Docs** <https://kafka.apache.org/>



Obrigado



Marcio Junior Vieira

CEO | Data Scientist | Palestrante |
Pesquisador | Professor



Marcio Junior Vieira
marcio@ambientelivre.com.br

Redes:

@ambientelivre

@ambientelivreopensource

Linkedin....: <https://www.linkedin.com/in/mvieira1/>

Slide da Palestra serão publicada no github e LinkedIn

Github: @ambientelivre